Partition Problem - Interval Partitioning

Objective: Allocate overlapping intervals (such as lectures) to the minimum number of rooms/resources so that no two overlapping intervals use the same room.

Brief Explanation

In the interval partitioning problem, every interval has a start time and a finish time. If two intervals overlap, they cannot be assigned to the same room. The goal is therefore to partition all intervals into the smallest possible number of groups (rooms), where intervals inside each group do not overlap.

The given C++ program uses a **greedy strategy with a min-heap**. First, all intervals are sorted according to their starting times. The min-heap stores the finishing times of lectures currently occupying rooms. For each new interval, the program checks the smallest finishing time. If that lecture has already finished before or exactly when the new lecture starts, that room is reused; otherwise, a new room is required. The finishing time of the current interval is then inserted into the heap.

Algorithm Steps

1. Sort all intervals by their start time.
2. Create a min-heap to store room finishing times.
3. For every interval, compare its start time with the earliest finishing room.
4. If the earliest room is free, remove its old finish time and reuse that room.
5. Insert the current interval's finish time into the heap.
6. After all intervals are processed, the heap size gives the minimum number of rooms required.

Complexity Analysis

Operation	Complexity	Reason
Sorting intervals	O(n log n)	n intervals are sorted by starting time.
Heap processing	O(n log n)	Each interval may perform heap pop/push operations.
Overall time	O(n log n)	Sorting and heap operations dominate.
Space	O(n)	In the worst case, all intervals overlap and the heap stores n finish times.

For the shown input: the algorithm prints the minimum number of rooms needed after processing all overlapping lecture intervals.

C++ Implementation

interval.cpp > main()

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <queue>
using namespace std;

int main() {
    vector<pair<int, int>> intervals = {
        {9, 10}, {9, 11}, {10, 12},
        {11, 12}, {9, 12}
    };

    // Sort by start time
    sort(intervals.begin(), intervals.end());

    // Min-heap stores room finishing times
    priority_queue<int, vector<int>, greater<int>> rooms;

    for (auto interval : intervals) {
        int start = interval.first;
        int finish = interval.second;

        // Reuse room if its lecture has finished
        if (!rooms.empty() && rooms.top() <= start) {
            rooms.pop();
        }

        rooms.push(finish);
    }

    cout << "Minimum rooms required: "
         << rooms.size() << endl;

    return 0;
}
```

Figure: C++ implementation using sorting and a min-heap (priority queue).